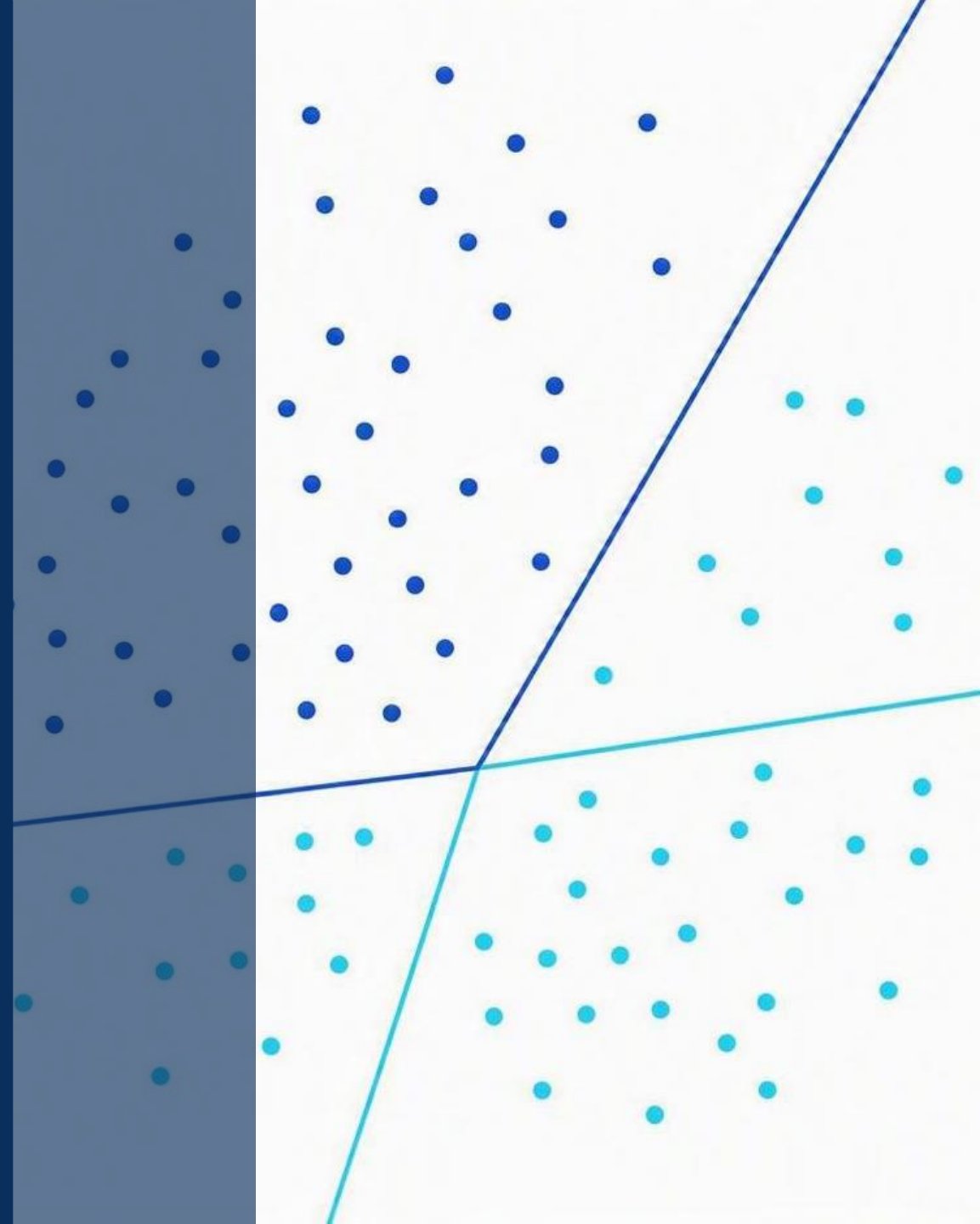


MACHINE LEARNING · GUÍA PRÁCTICA

Algoritmos de clasificación en Machine Learning

Cómo funcionan, cuándo usarlos y cómo compararlos

Orientado a quienes ya conocen los fundamentos de ML



¿Qué es un problema de clasificación?

La clasificación busca predecir a qué categoría o clase pertenece cada observación a partir de sus variables.

Clasificación binaria

Dos clases posibles

- Spam / no spam
- Cliente que abandona / no abandona
- Diagnóstico positivo / negativo

Clasificación multiclase

Tres o más clases posibles

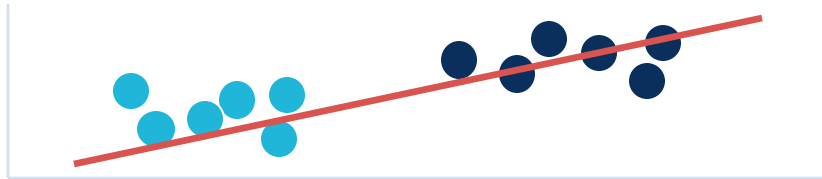
- Tipo de flor Iris (Setosa, Versicolor, Virginica)
- Categoría de un producto
- Nivel de riesgo: bajo / medio / alto

Clasificación vs. regresión

Ambas son aprendizaje supervisado, pero difieren en el tipo de salida que predicen.

Clasificación

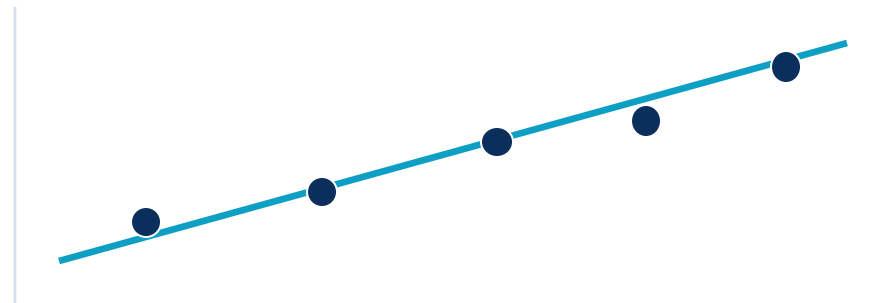
Predice una clase o etiqueta



Ej.: predecir la especie de una flor

Regresión

Predice un valor numérico continuo



Ej.: predecir el precio de una casa

Los algoritmos que vamos a ver

1

Regresión logística

Baseline lineal e interpretable

2

Árboles de decisión

Reglas explicables paso a paso

3

Random Forest

Muchos árboles que votan

4

SVM

Máximo margen entre clases

5

Naive Bayes

Probabilidades del teorema de Bayes

6

Redes neuronales (MLP)

No linealidad flexible

7

Gradient Boosting

XGBoost y LightGBM

8

LDA

Clásico + reducción de dimensión

Criterios para elegir un algoritmo

No existe un único algoritmo mejor: la elección depende de estos factores.

Tamaño del dataset

¿Pocos o muchos ejemplos?

Número de variables

Dimensionalidad del problema

Tipo de variables

Continuas, categóricas, texto

Linealidad

¿Frontera lineal o curva?

Interpretabilidad

¿Hay que explicar el modelo?

Velocidad de entrenamiento

Tiempo y recursos disponibles

Precisión esperada

Nivel de accuracy necesario

Riesgo de overfitting

Capacidad de generalizar

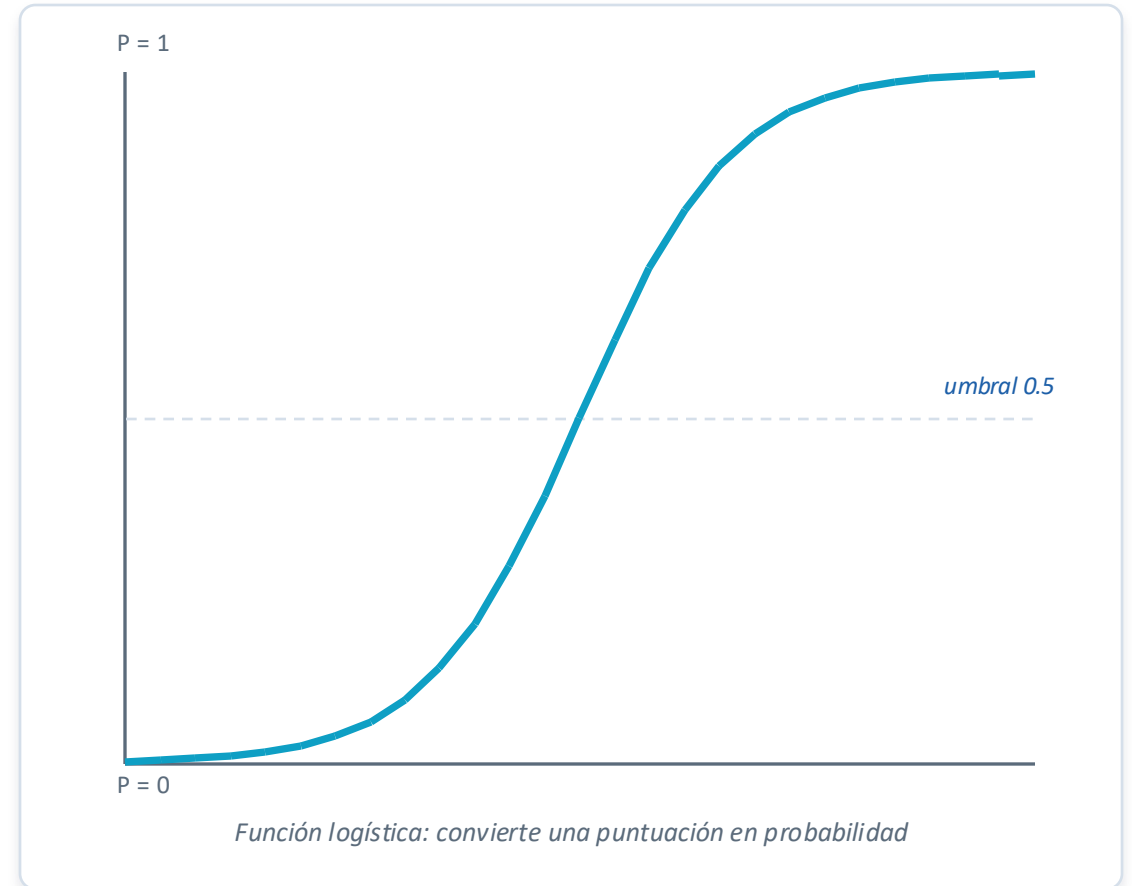
Explicar al negocio

Confianza y transparencia

Regresión logística: idea general

Aunque se llame "regresión", es un algoritmo de clasificación. Estima la probabilidad de que una observación pertenezca a una clase.

- En clasificación binaria decide entre dos clases
- En multiclase se usa como regresión logística multinomial
- Salida interpretable como probabilidad (0 a 1)



¿Cómo funciona la regresión logística?

1

Combina las variables

Calcula una suma ponderada de las variables de entrada (pesos).

2

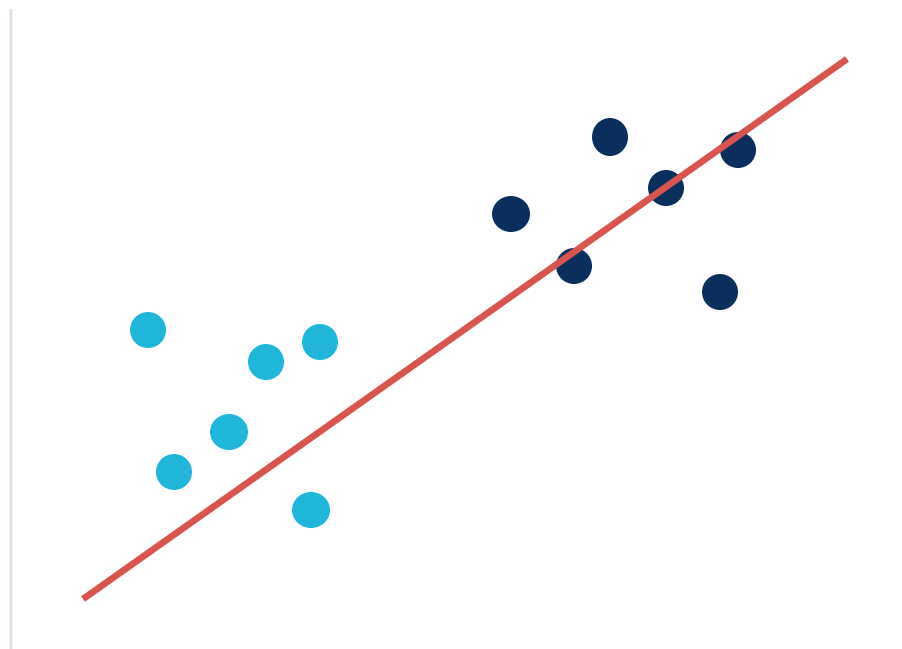
Convierte en probabilidad

La función logística transforma esa puntuación en un valor entre 0 y 1.

3

Aplica una frontera

Si la probabilidad supera el umbral, asigna la clase.



Frontera de decisión que separa las dos clases

¿Cuándo usar regresión logística?

Úsalo cuando...

- Necesitas un modelo simple e interpretable
- La relación variables–clase es aproximadamente lineal
- Quieres explicar el peso de cada variable
- Buscas una línea base sólida para comparar

Ejemplos

- Predicción de churn (abandono de clientes)
- Riesgo crediticio (aprobar / rechazar)
- Clasificación médica binaria
- Clasificación multiclase simple

Ventajas y limitaciones

Ventajas

- Rápida de entrenar y de aplicar
- Fácil de interpretar (coeficientes)
- Excelente baseline de referencia
- Funciona bien en datasets medianos o grandes

Limitaciones

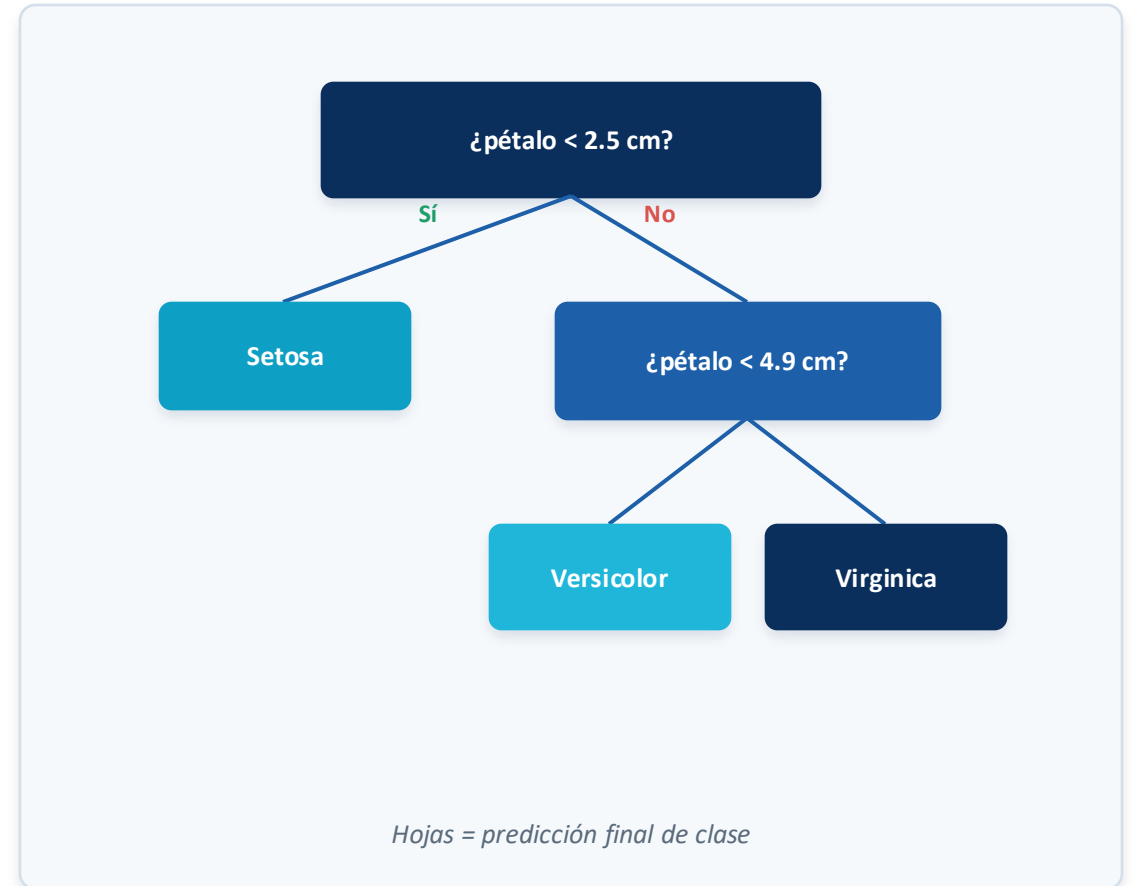
- No capta relaciones no lineales sin transformar variables
- Se queda corta si las clases no son separables linealmente
- Sensible a variables muy correlacionadas

En resumen: Primer modelo a probar casi siempre; si el problema es lineal, puede ser suficiente.

Árboles de decisión: idea general

Un árbol clasifica tomando decisiones sucesivas: preguntas simples sobre las variables que van dividiendo los datos.

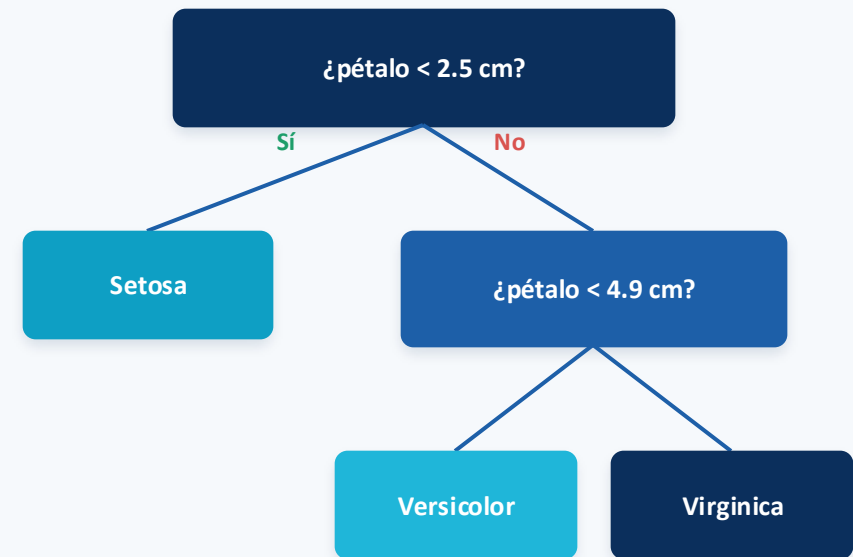
- "Si el pétalo mide menos que X $\rightarrow$ clase A; si no, evaluar otra condición"
- El razonamiento es visual y fácil de seguir
- Ideal para datos tabulares y reglas de negocio



¿Cómo funciona un árbol de decisión?

El proceso, paso a paso

- Divide el dataset en ramas según las variables
- Busca en cada paso la variable que mejor separa las clases
- Usa criterios como Gini o entropía para medir la pureza
- Las hojas representan la predicción final de clase



Hojas = predicción final de clase

¿Cuándo usar árboles de decisión?

Úsalo cuando...

- Necesitas interpretabilidad y transparencia
- Hay relaciones no lineales entre variables
- Quieres explicar el razonamiento del modelo
- Trabajas con datos tabulares
- Necesitas mostrar cómo se toman las decisiones

Ejemplos

- Dataset Iris: permite visualizar reglas simples
- Segmentación de clientes con reglas claras
- Aprobación de solicitudes con criterios explícitos
- Diagnóstico guiado por preguntas

Ventajas y limitaciones

Ventajas

- Muy interpretables y visuales
- Capturan relaciones no lineales
- Requieren poco preprocesamiento
- No necesitan escalar las variables

Limitaciones

- Sobreajustan con facilidad
- Sensibles a pequeños cambios en los datos
- Un árbol muy profundo generaliza mal

En resumen: Un solo árbol es didáctico; para producción, conviene combinarlos (Random Forest o boosting).

Random Forest: idea general

Random Forest combina muchos árboles de decisión. Cada árbol vota una clase y el bosque decide por mayoría.

- Muchos árboles diversos reducen el error de uno solo
- La "sabiduría del grupo" mejora la robustez
- Menos sobreajuste que un árbol individual



¿Cómo funciona Random Forest?

Bagging + aleatoriedad

- Entrena muchos árboles sobre subconjuntos distintos de datos
- Cada árbol ve solo una parte de las variables (aleatoriedad)
- Cada árbol predice una clase de forma independiente
- La predicción final se obtiene por votación mayoritaria



El bosque decide combinando muchos árboles

¿Cuándo usar Random Forest?

Úsalo cuando...

- Buscas buen rendimiento sin mucho ajuste fino
- Trabajas con datos tabulares
- Quieres reducir el overfitting de un único árbol
- Necesitas importancia de variables
- Priorizas precisión razonable con robustez

Ejemplos

- Scoring de clientes y detección de fraude
- Clasificación tabular de propósito general
- Problemas con muchas variables mezcladas
- Primera opción sólida "que casi siempre funciona"

Ojo: En datasets extremadamente pequeños no siempre es la mejor opción frente a modelos clásicos.

Ventajas y limitaciones

Ventajas

- Robusto y fiable en muchos problemas
- Reduce el overfitting frente a un solo árbol
- Maneja relaciones no lineales
- Ofrece importancia de variables

Limitaciones

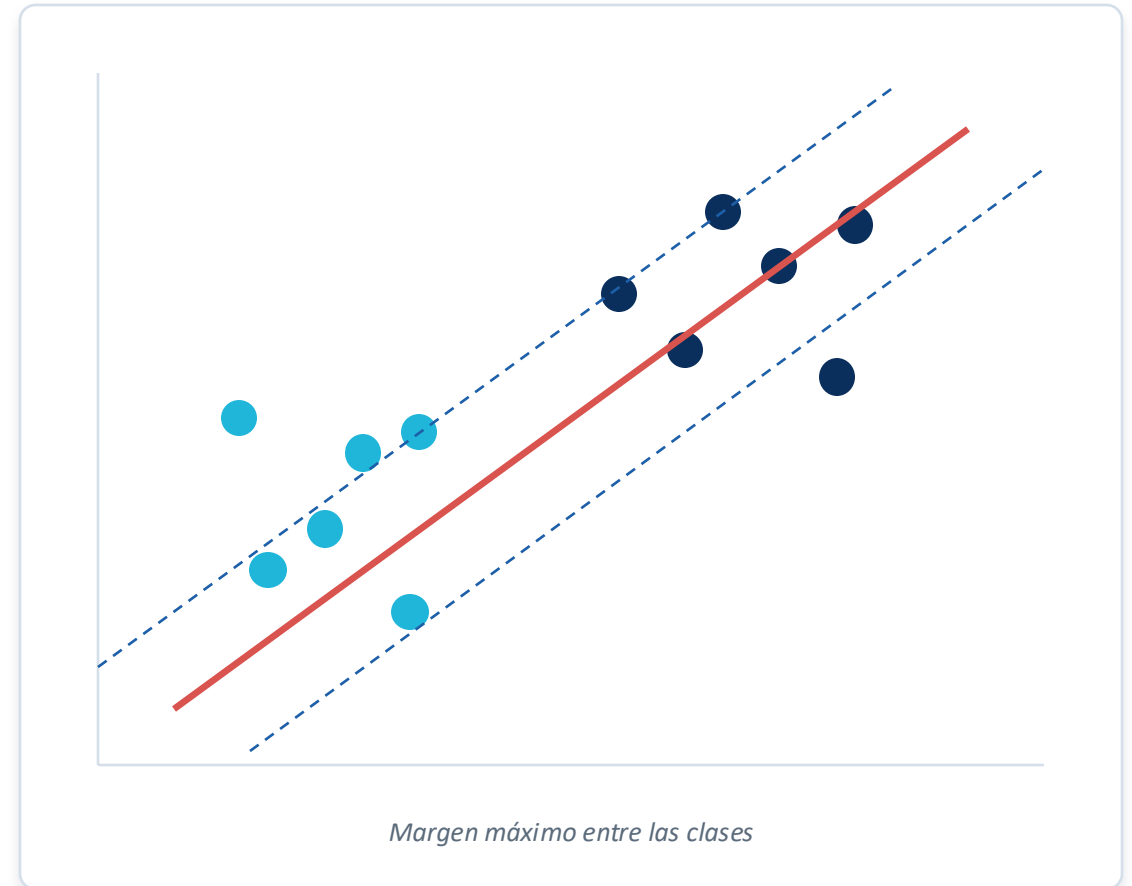
- Menos interpretable que un árbol simple
- Más pesado computacionalmente
- No siempre el mejor con datasets muy pequeños

En resumen: Gran opción por defecto para datos tabulares cuando la interpretabilidad no es crítica.

SVM: idea general

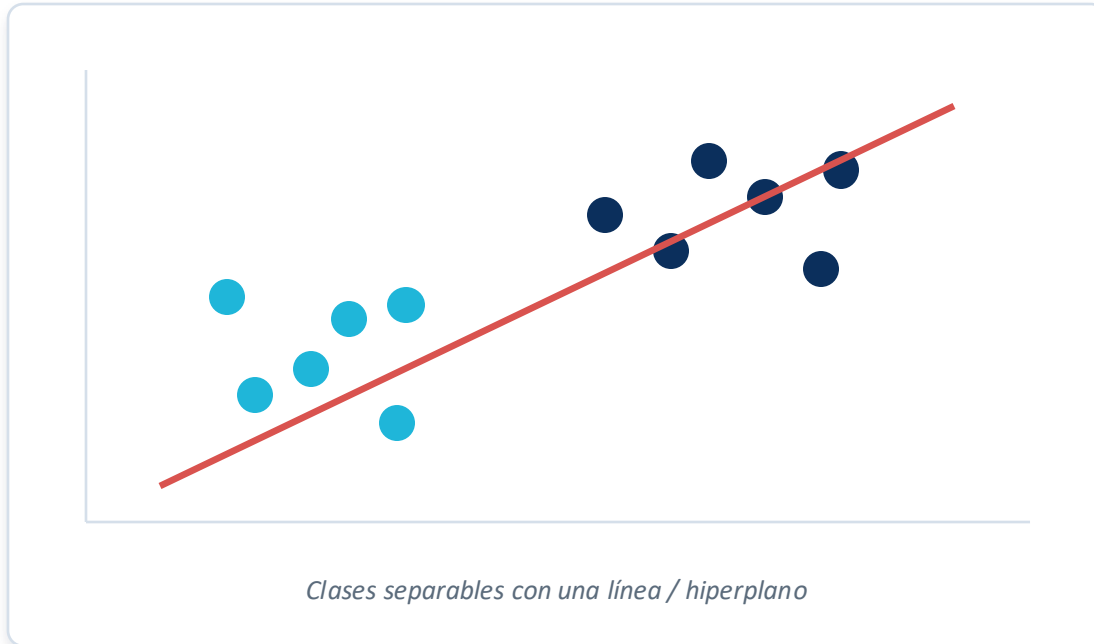
Support Vector Machine busca la mejor frontera de separación entre clases. La idea central: encontrar el margen máximo entre ellas.

- Maximiza la distancia a los puntos más cercanos
- Esos puntos críticos son los vectores de soporte
- Una frontera con margen amplio generaliza mejor



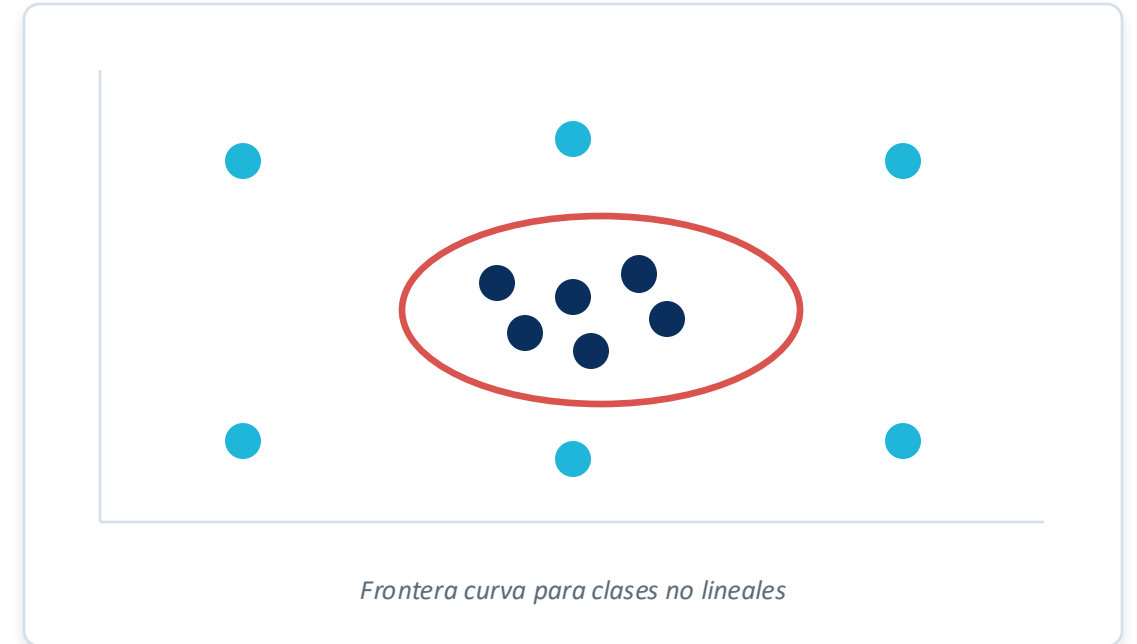
SVM con kernel lineal y kernel RBF

Kernel lineal



Útil cuando las clases se separan aproximadamente con una recta.

Kernel RBF



Útil cuando la frontera entre clases es curva o no lineal.

¿Cuándo usar SVM?

Úsalo cuando...

- Tienes datasets pequeños o medianos
- Hay muchas variables (alta dimensionalidad)
- Necesitas una frontera de decisión potente
- Las clases están bien separadas
- Puedes escalar correctamente la información

Ejemplos

- Clasificación de textos y documentos
- Bioinformática con muchas variables
- Reconocimiento de patrones con pocos datos
- Problemas con frontera no lineal (kernel RBF)

Ojo: SVM suele requerir escalado de variables para funcionar bien.

Ventajas y limitaciones

Ventajas

- Muy potente en datasets pequeños o medianos
- Funciona bien con alta dimensionalidad
- El kernel RBF captura no linealidad
- Frontera de decisión de margen máximo

Limitaciones

- Lento en datasets grandes
- Poco interpretable
- Requiere ajustar bien C y gamma
- Necesita escalado de variables

En resumen: Elección fuerte con pocos datos y muchas variables, a cambio de más trabajo de ajuste.

Naive Bayes: idea general

Naive Bayes se basa en probabilidades y en el teorema de Bayes. Estima qué clase es más probable dado lo observado.

- Asume que las variables son independientes entre sí
- En la práctica esa suposición no siempre se cumple...
- ...pero aun así suele funcionar sorprendentemente bien

Teorema de Bayes

$$P(\text{clase} \mid \text{datos}) \propto P(\text{datos} \mid \text{clase}) \cdot P(\text{clase})$$

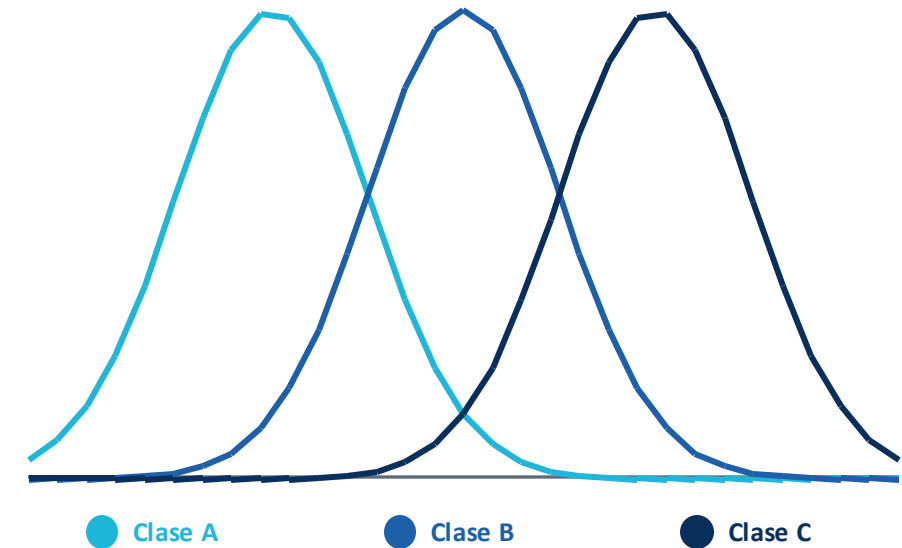
Se calcula la probabilidad de cada clase y se elige la más alta.

"Naive" (ingenuo) = supone independencia entre variables

Gaussian Naive Bayes

La variante Gaussian NB se usa cuando las variables (features) son continuas.

- Modela cada variable con una distribución normal por clase
- Adecuado para datasets como Iris, con variables numéricas continuas
- Para texto se usan otras variantes: Multinomial NB o Bernoulli NB



Cada variable continua se modela con una distribución normal por clase

¿Cuándo usar Naive Bayes?

Úsalo cuando...

- Necesitas un modelo muy rápido
- Quieres una línea base simple
- Tienes pocos datos
- Las variables son aproximadamente independientes
- Trabajas con texto (Multinomial o Bernoulli NB)

Ejemplos

- Filtrado de spam y clasificación de correo
- Análisis de sentimiento en texto
- Categorización de documentos
- Iris y variables continuas → Gaussian NB

Ojo: Para variables continuas usa Gaussian NB; para texto, Multinomial o Bernoulli NB.

Ventajas y limitaciones

Ventajas

- Muy rápido de entrenar y predecir
- Funciona bien con pocos datos
- Fácil de implementar
- Buena baseline, sobre todo en texto

Limitaciones

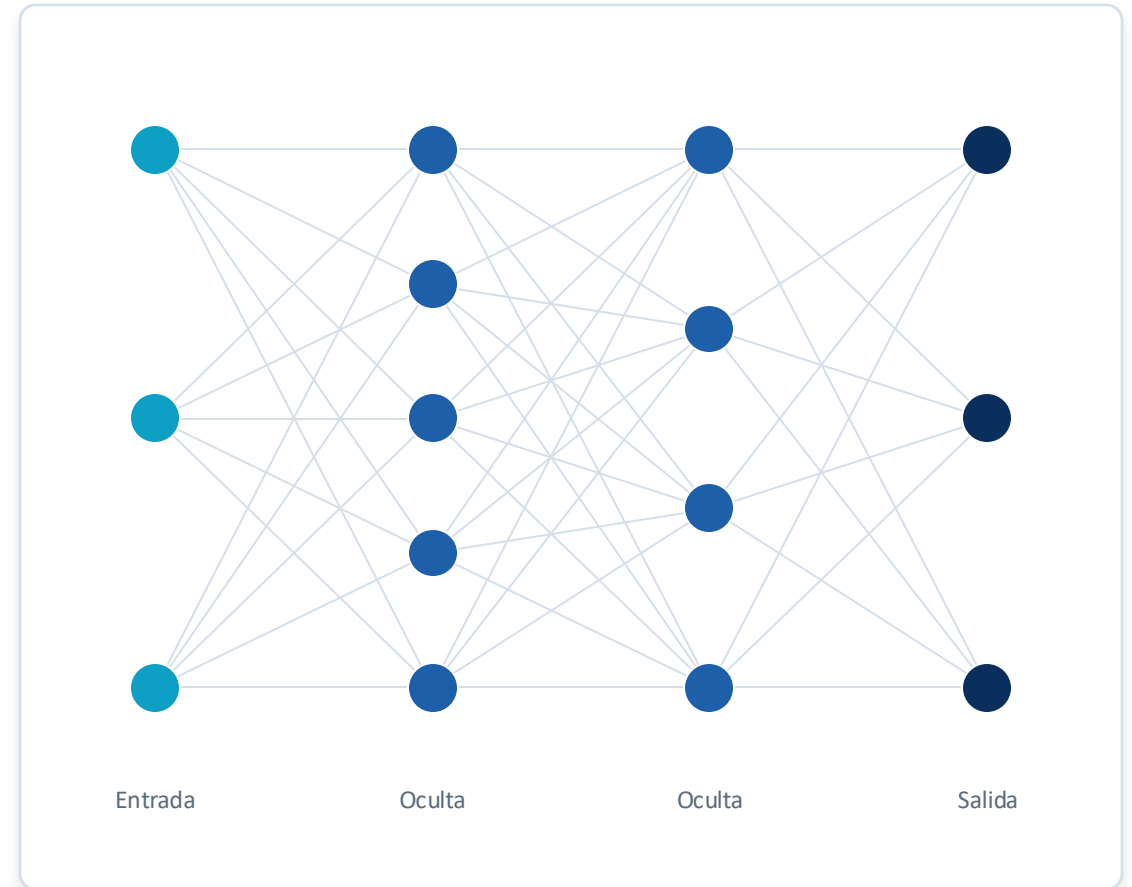
- La suposición de independencia puede ser demasiado fuerte
- Se queda corto con relaciones complejas entre variables
- Menos flexible que otros algoritmos

En resumen: Ideal como referencia rápida; competitivo en texto pese a su simplicidad.

MLP: idea general

Un MLP (perceptrón multicapa) es una red neuronal feedforward con capas de neuronas. Puede aprender relaciones no lineales complejas.

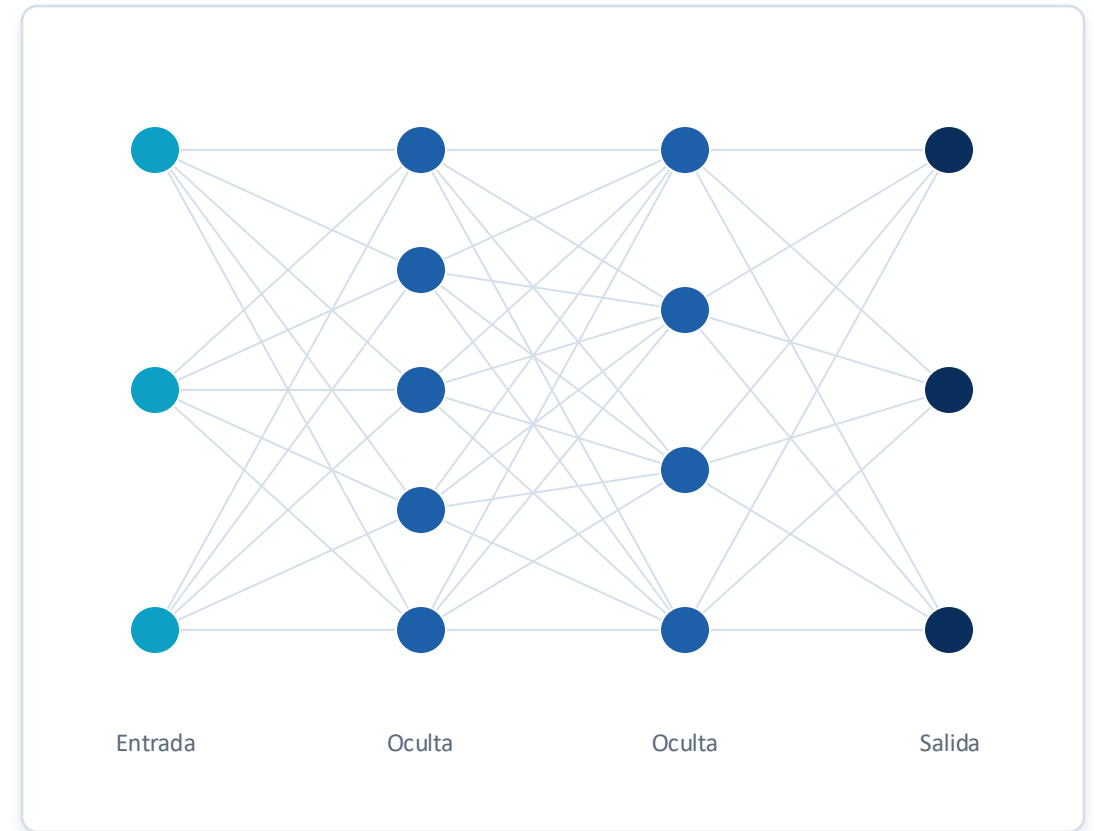
- Las neuronas se organizan en capas conectadas
- Aprende representaciones a partir de los datos
- Es la base conceptual del deep learning



¿Cómo funciona un MLP?

Flujo hacia adelante y ajuste

- Recibe las variables de entrada
- Las procesa a través de una o varias capas ocultas
- Aplica funciones de activación no lineales
- Ajusta los pesos durante el entrenamiento (backpropagation)



¿Cuándo usar MLP?

Úsalo cuando...

- Hay relaciones no lineales complejas
- Dispones de suficiente cantidad de datos
- Buscas flexibilidad para modelar patrones difíciles
- La interpretabilidad no es la prioridad principal

Ejemplos

- Reconocimiento de patrones complejos
- Problemas con muchas interacciones entre variables
- Antesala de arquitecturas de deep learning
- Iris: funciona, pero es "matar una mosca a cañonazos"

Ojo: Para datasets pequeños suele ser innecesario frente a modelos más simples.

Ventajas y limitaciones

Ventajas

- Muy flexible
- Modela relaciones complejas y no lineales
- Base conceptual del deep learning

Limitaciones

- Requiere más datos
- Necesita más ajuste de hiperparámetros
- Poco interpretable
- Innecesario para problemas simples o datasets pequeños

En resumen: Potente cuando sobran datos y complejidad; sobredimensionado para casos sencillos.

Gradient Boosting: idea general

Gradient Boosting combina modelos débiles —normalmente árboles pequeños— de forma secuencial. Cada nuevo modelo corrige los errores del anterior.

- Aprende de los errores paso a paso
- Suma muchos modelos débiles en uno fuerte
- Muy competitivo en datos tabulares



XGBoost y LightGBM

XGBoost

- Implementación muy potente y ampliamente adoptada
- Referencia habitual en competiciones de datos
- Gran control mediante regularización

LightGBM

- Implementación muy eficiente y rápida
- Destaca especialmente en datasets grandes
- Menor consumo de memoria en el entrenamiento

En común: ambas son variantes de gradient boosting muy competitivas en datos tabulares, sin entrar en detalles internos.

¿Cuándo usar Gradient Boosting?

Úsalo cuando...

- Buscas alto rendimiento predictivo
- Trabajas con datos tabulares
- Quieres competir por accuracy
- Dispones de tiempo para el ajuste de hiperparámetros

Ejemplos

- Competiciones de ciencia de datos
- Scoring, riesgo y detección de fraude
- Problemas tabulares donde importa la precisión
- Grandes volúmenes de datos (LightGBM)

Ojo: Para un dataset de 150 filas puede funcionar, pero probablemente sea overkill.

Ventajas y limitaciones

Ventajas

- Muy buen rendimiento en problemas reales
- Captura relaciones no lineales
- Referencia en competiciones y proyectos tabulares

Limitaciones

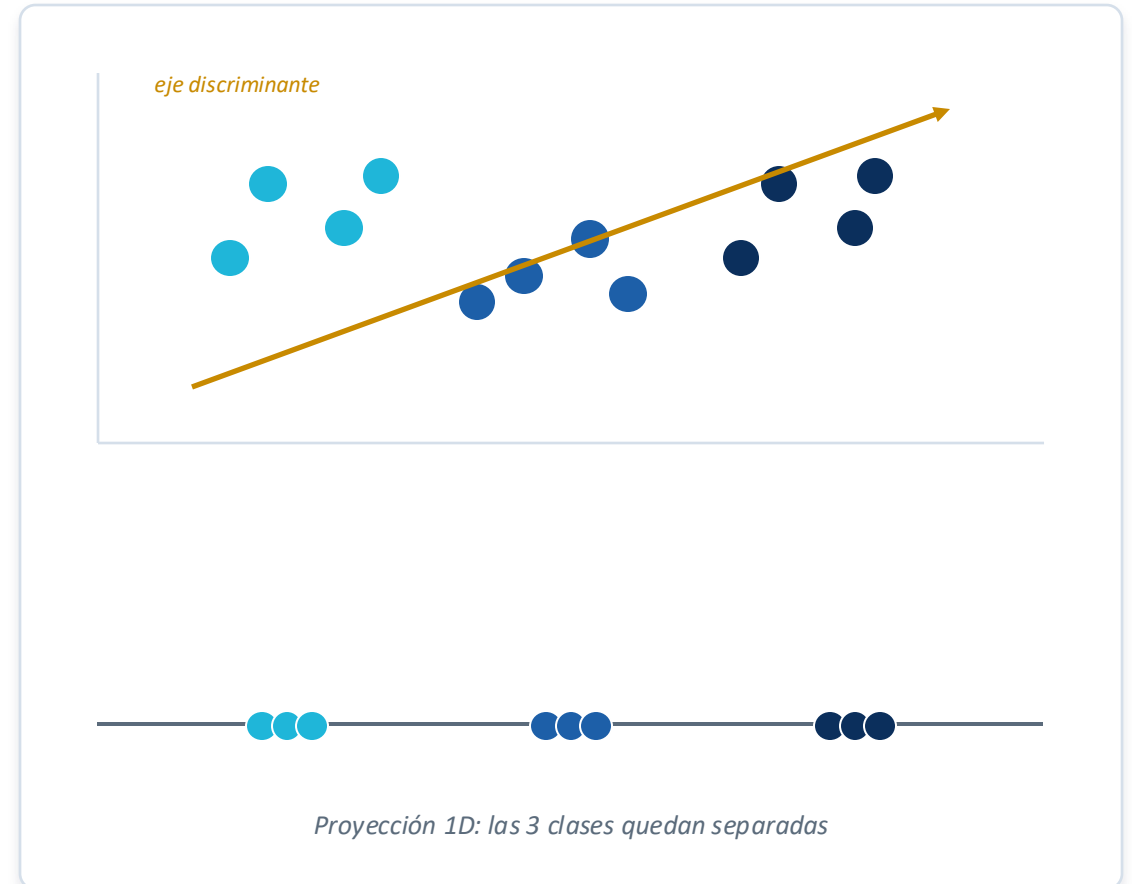
- Más complejo de ajustar
- Puede sobreajustar si no se controla
- Menos interpretable
- Excesivo para problemas simples

En resumen: Máximo rendimiento en tabular a cambio de complejidad; no siempre justificado en datos pequeños.

LDA: idea general

Linear Discriminant Analysis es un método clásico de clasificación y también de reducción de dimensionalidad supervisada.

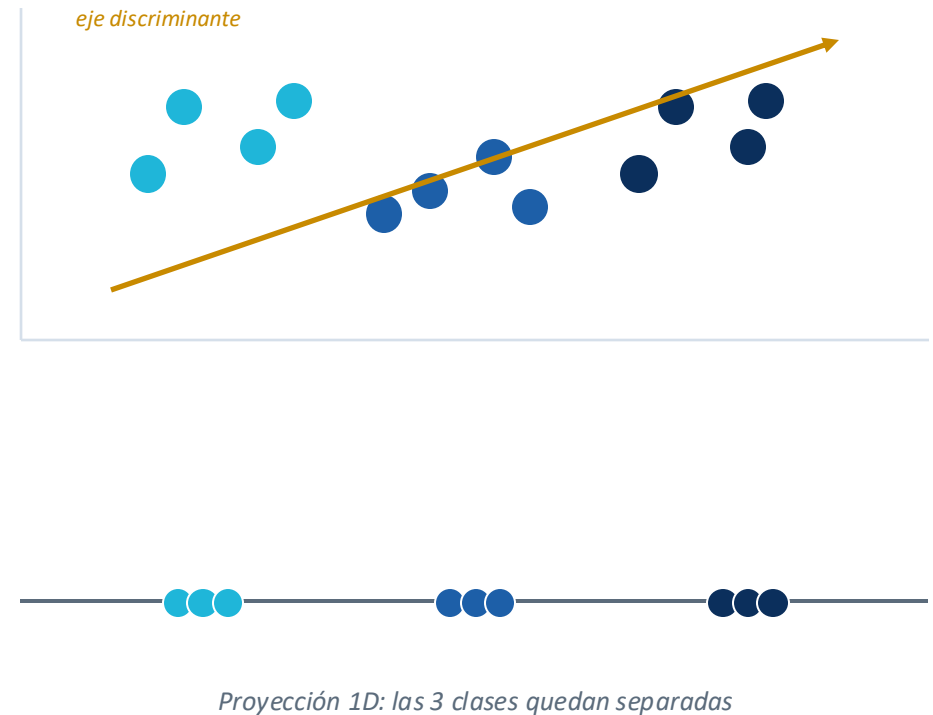
- Proyecta los datos para separar al máximo las clases
- Combina clasificación y reducción de dimensión
- Un clásico muy didáctico, ideal para Iris



¿Cómo funciona LDA?

Separar clases, compactar grupos

- Maximiza la separación entre las clases
- Minimiza la variabilidad dentro de cada clase
- Genera ejes discriminantes (nuevas direcciones)
- Proyecta los datos manteniendo la separación



¿Cuándo usar LDA?

Úsalo cuando...

- Las clases tienen distribuciones aproximadamente normales
- Las clases comparten una covarianza similar
- Necesitas un modelo clásico, interpretable y eficiente
- Quieres reducir dimensión manteniendo la separación

Ejemplos

- Iris: caso clásico para enseñar LDA
- Reducción de dimensión previa a otro modelo
- Reconocimiento de rostros (eigen/fisherfaces)
- Problemas con clases bien estructuradas

Ojo: LDA es un clásico para Iris: sencillo, rápido y muy visual.

Ventajas y limitaciones

Ventajas

- Rápido y eficiente
- Interpretable
- Útil para reducción de dimensión supervisada
- Muy didáctico para datasets como Iris

Limitaciones

- Asume cierta estructura estadística en los datos
- Falla con fronteras muy no lineales
- Menos flexible que árboles o boosting

En resumen: Clásico elegante para datos bien estructurados y con fines didácticos.

Comparativa general de algoritmos

Algoritmo	Mejor caso de uso	Interpret.	No lineal.	Overfitting	Escalado	Datos peq.	Comentario práctico
Regresión logística	Baseline lineal	Alta	Baja	Bajo	Sí	Sí	Punto de partida sólido
Árbol de decisión	Reglas explicables	Alta	Media	Alto	No	Medio	Interpretable, sobreajusta
Random Forest	Tabular robusto	Media	Alta	Bajo	No	Medio	Sólido casi siempre
SVM	Pocos datos, alta dim.	Baja	Alta	Medio	Sí	Sí	Potente, sensible al tuning
Gaussian NB	Baseline rápido	Media	Baja	Bajo	No	Sí	Muy rápido, pocos datos
MLP	No linealidad compleja	Baja	Alta	Alto	Sí	No	Requiere datos y tuning
Gradient Boosting	Máx. accuracy tabular	Baja	Alta	Medio	No	Parc.	Top en tabular, ajuste fino
LDA	Clásico + reduc. dim.	Alta	Baja	Bajo	Sí	Sí	Excelente para Iris

Interpretabilidad y overfitting: verde = favorable · rojo = desfavorable · dorado = intermedio

¿Qué algoritmo elegir según el escenario?

Si... Necesito interpretabilidad

Regresión logística · Árbol · LDA

Si... Quiero una baseline rápida

Regresión logística · Naive Bayes

Si... Tengo relaciones no lineales

Árbol · Random Forest · SVM RBF · Boosting · MLP

Si... Tengo pocos datos

LDA · Naive Bayes · SVM · Árbol simple

Si... Alto rendimiento en tabular

Random Forest · Gradient Boosting

Si... Dataset muy pequeño (tipo Iris)

Modelos clásicos; evitar excesiva complejidad

Conclusión

1

La elección depende del contexto

El mejor algoritmo depende del problema, los datos y el objetivo.

2

Más complejo $\neq$ mejor

El modelo más sofisticado no siempre es el más adecuado.

3

Comparar es aprender

Para enseñar o empezar, conviene comparar modelos simples y complejos.

4

Los clásicos brillan con pocos datos

LDA, regresión logística, árboles, SVM o Gaussian NB pueden ser excelentes.

5

Cuidado con el overkill — MLP, XGBoost o LightGBM funcionan, pero suelen sobrar en datasets pequeños.

IDEA PRINCIPAL

En clasificación, elegir bien el algoritmo no significa elegir el más sofisticado, sino el más adecuado para el problema, los datos y la necesidad de interpretación.